COMPTE RENDU TP7

Flexbox, Grid et Responsive Design

Organisation d’une page Web adaptable aux écrans mobile, tablette et ordinateur

Nom et prénom	Ranim Ben Arfi
Classe / Groupe	1ère année Technologie de l’informatique
Année universitaire	2026 / 2027
Module	Développement Web
Travail	TP7 — Flexbox, Grid et Responsive Design
Date	12/10/2026

Atelier Développement Web 2026/2027

1. Objectifs du TP

Ce TP a pour objectif d'organiser une interface Web avec Flexbox et CSS Grid, puis de l'adapter aux différentes tailles d'écran grâce au Responsive Design et aux Media Queries.

• Utiliser Flexbox pour aligner et répartir les éléments.

• Comprendre justify-content, align-items, gap et flex-wrap.

• Créer une grille avec grid-template-columns, grid-template-rows et grid-column.

• Comprendre la différence entre Flexbox et Grid.

• Appliquer l'approche Mobile First et tester avec DevTools.

• Adapter la page pour mobile, tablette et desktop.

2. Structure du projet

Le projet comprend la page d'accueil index.html, la feuille de style style.css, la page pages/contact.html et le dossier images/. L'illustration du campus est dessinée en CSS dans cette version afin que la page reste autonome et s'affiche sans image externe.

```
TP_Flexbox_Grid_Responsive/
■■■ index.html
■■■ style.css
■■■ pages/
■    ■■■ contact.html
■■■ images/
```

3. Réalisation du TP

3.1 Page d'accueil HTML

La page contient un header et sa navigation, une section hero avec texte et illustration, quatre cartes de formation, trois projets et un footer.

<!DOCTYPE html><html lang="fr"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Campus Web</title>
<link rel="stylesheet" href="style.css"></head>
<body><header><h1>Campus Web</h1><nav>AccueilContactÀ proposProjets</nav></header><main><section class="hero"> <h2>Bienvenue sur Campus Web</h2> Découvrez nos formations et nos projets dans le domaine du développement Web. Découvrir <img src="pho.jpg" alt="Campus"> </section><section class="formations"><h2>Nos formations</h2> <article class="card"><h3>HTML5</h3> Apprendre à structurer une page Web. En savoir plus</article><article class="card"><h3>CSS3</h3> Créer des interfaces modernes. En savoir plus</article><article class="card"><h3>JavaScript</h3> Ajouter des interactions aux pages Web. En savoir plus</article><article class="card"><h3>PHP</h3> Développer des applications Web côté serveur. En savoir plus</article> </section><section class="projets"><h2>Nos projets</h2> <article class="projet"><h3>Projet 1</h3> Création d'un site Web. </article><article class="projet"><h3>Projet 2</h3> Création d'une application Web. </article><article class="projet"><h3>Projet 3</h3> Création d'une interface responsive. </article> </section></main><footer> 2026 - Campus Web </footer></body></html>

3.2 Organisation du header avec Flexbox

display: flex active Flexbox. justify-content: space-between sépare le titre de la navigation, align-items: center aligne verticalement les éléments et gap définit l'espace entre les liens.

```
nav { display: flex; gap: 28px; }
nav a:hover { color: orange; }
 nav {   display: flex;  gap: 25px; }
  nav a { color: white; }
header, footer {
display: flex;
justify-content: space-between;
align-items: center;   padding: 20px 40px; background-color: rgb(57, 57, 222);   color: #f5f5f5;
}
```

3.3 Section Hero et cartes

La section hero place le texte et l'image côte à côte. Les cartes utilisent flex-wrap pour passer à la ligne lorsque la largeur disponible diminue.

```
.hero {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 55px;
  padding: 65px 10px;
}
.hero-text, .hero-image { width: 50%; }
.cards {
  display: flex;
  gap: 18px;
  flex-wrap: wrap;
}
.card { flex: 1; min-width: 210px; }
```

3.4 Organisation des projets avec Grid

CSS Grid organise les projets en lignes et en colonnes. repeat(3, 1fr) crée trois colonnes de même largeur. grid-column: span 2 permet au premier projet d'occuper deux colonnes.

```
.projet-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 18px;
}
.projet:first-child { grid-column: span 2; }
```

Propriété	Rôle
display: flex	Active Flexbox pour organiser les éléments sur un axe.
flex-wrap	Autorise le passage des éléments à la ligne.
display: grid	Active une mise en page en lignes et colonnes.
grid-template-columns	Définit le nombre et la largeur des colonnes.
grid-column: span 2	Fait occuper deux colonnes à un élément.

4. Responsive Design

Le Responsive Design adapte la mise en page à la largeur de l'écran. La page utilise trois plages : mobile jusqu'à 767 px, tablette de 768 à 1023 px et ordinateur à partir de 1024 px.

4.1 Version mobile — jusqu'à 767 px

```
@media screen and (max-width: 767px) {
  header { flex-direction: column; gap: 18px; }
  nav { flex-wrap: wrap; justify-content: center; }
  .hero { flex-direction: column; }
  .hero-text, .hero-image { width: 100%; }
  .cards { flex-direction: column; }
  .projet-container { grid-template-columns: 1fr; }
  .projet:first-child { grid-column: auto; }
}
```

4.2 Version tablette — 768 à 1023 px

```
@media screen and (min-width: 768px) and (max-width: 1023px) {
  .cards { display: grid; grid-template-columns: repeat(2, 1fr); }
  .projet-container { grid-template-columns: repeat(2, 1fr); }
}
```

4.3 Version desktop — à partir de 1024 px

```
@media screen and (min-width: 1024px) {
  .cards { display: flex; flex-direction: row; }
  .projet-container { grid-template-columns: repeat(3, 1fr); }
}
```

Pour tester avec DevTools : clic droit sur la page, choisir Inspecter puis activer Device Toolbar. Tester une largeur de téléphone, de tablette et d'ordinateur.

5. Résultat — Version ordinateur

La capture suivante présente le rendu de la page Campus Web sur un écran large. Les formations sont réparties en plusieurs cartes et les projets sont organisés avec Grid.

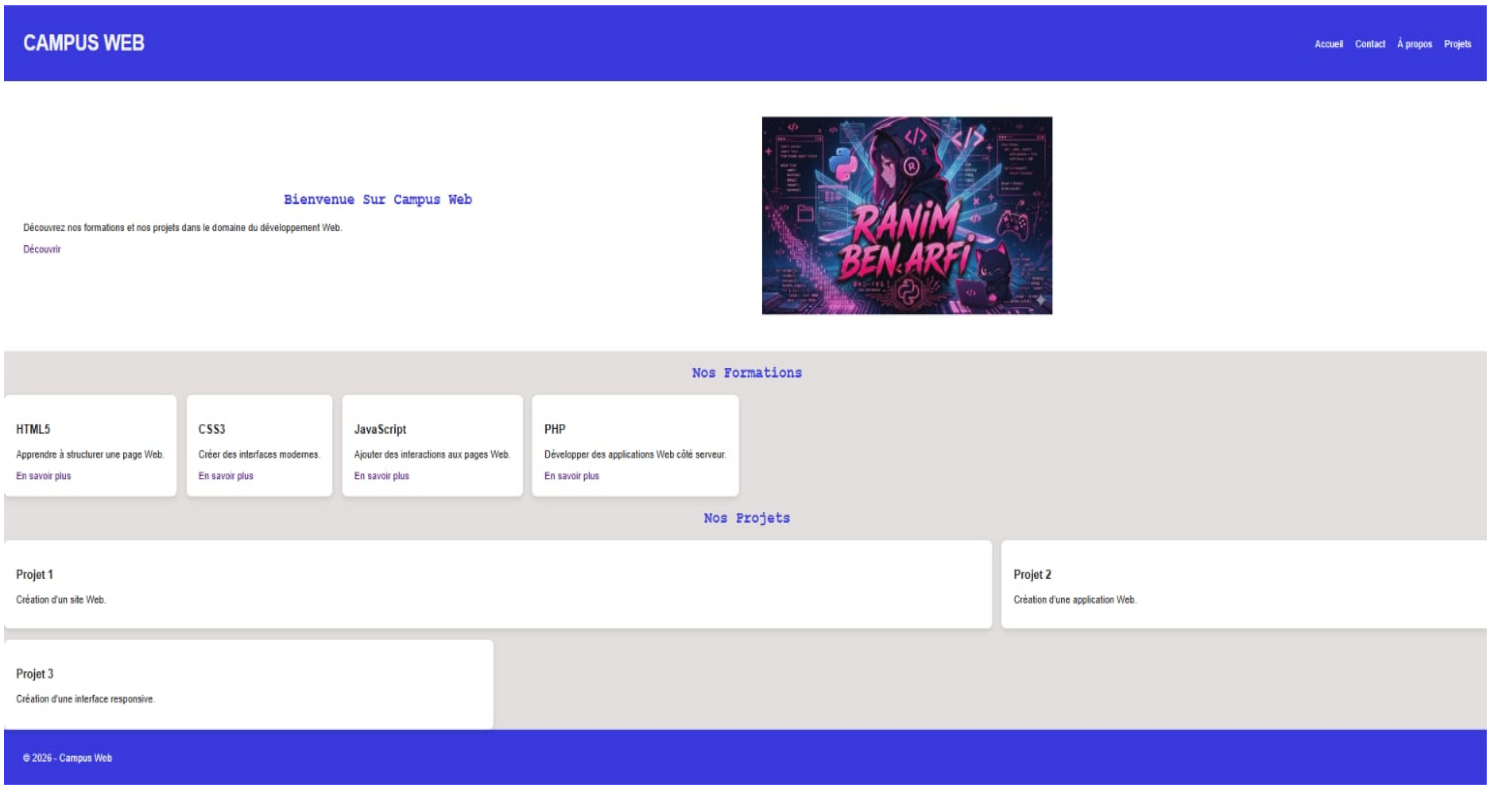


Figure 1 — Rendu de la page d’accueil sur ordinateur.

6. Résultat — Version mobile

Sur mobile, la navigation peut revenir à la ligne, le texte et l'illustration s'empilent, les formations sont disposées verticalement et les projets occupent une seule colonne.

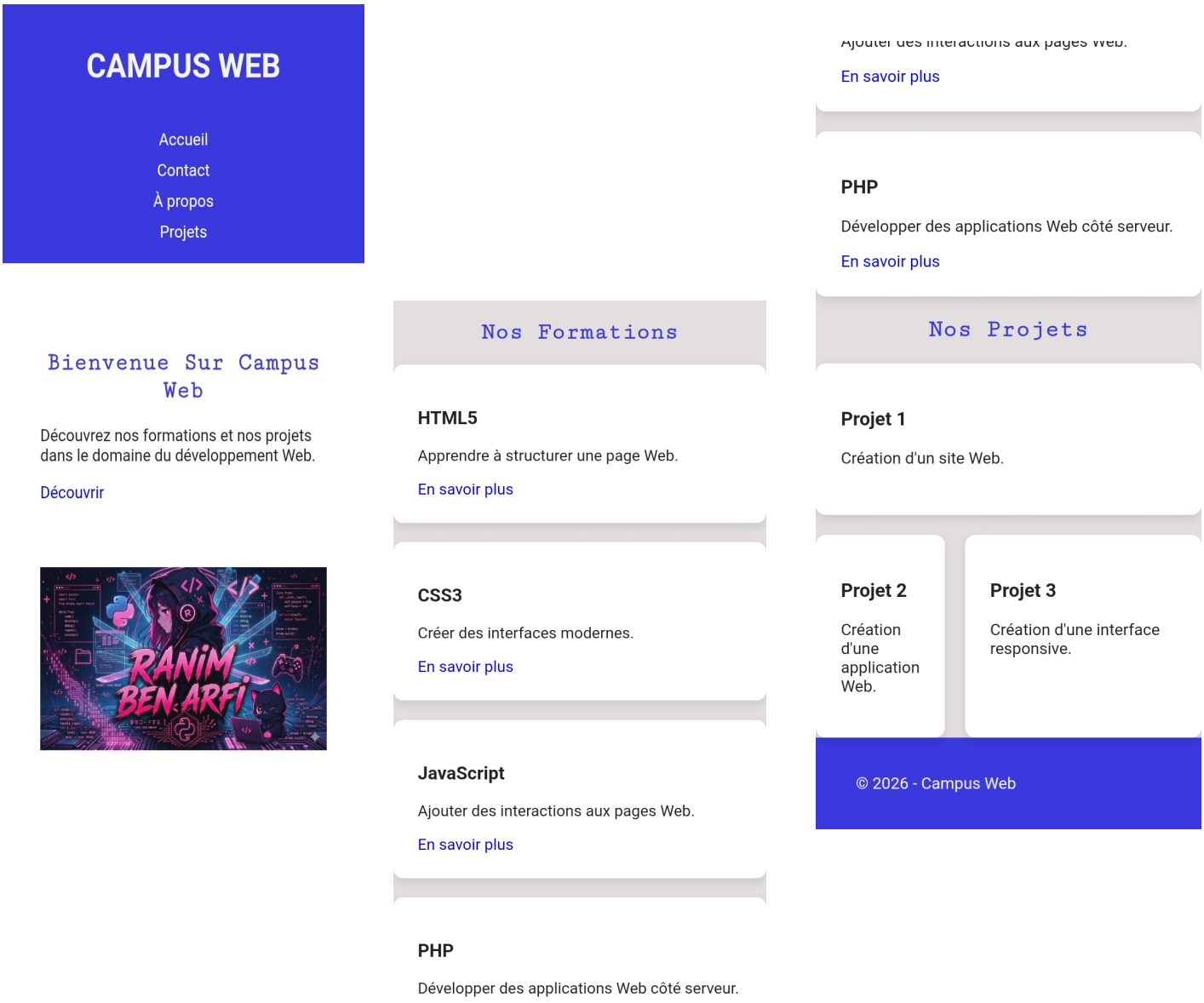


Figure 2 — Rendu responsive de la page d'accueil sur téléphone.

7. Page de contact

La page pages/contact.html contient le titre Contact, un lien de retour vers la page d'accueil et un bloc de présentation. Le chemin ../index.html remonte du dossier pages vers le dossier principal.

```
<!DOCTYPE html>
<html lang="fr">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact - Campus Web</title>
  <link rel="stylesheet" href="../style.css">
</head>
<body>
  <header>
    <h1>Contact</h1>
    <nav><a href="../index.html">Accueil</a></nav>
  </header>
  <main>
    <section class="formations">
      <h2>Nous contacter</h2>
      <p>Une question sur nos formations ou nos projets ?</p>
      <a href="../index.html">Retour à l'accueil</a>
    </section>
  </main>
  <footer><p>© 2026 - Campus Web</p></footer>
</body>
</html>
```

8. Observations et bonnes pratiques

• Flexbox convient surtout à une organisation sur un axe horizontal ou vertical.

• Grid convient à une organisation en lignes et colonnes.

• Les Media Queries appliquent des règles selon la largeur de l'écran.

• Les images doivent rester dans la largeur de leur conteneur.

• Le test avec DevTools permet de vérifier plusieurs tailles d'écran.

9. Compétences acquises

• Organiser une navigation et une section d'accueil avec Flexbox.

• Créer des cartes et une grille de projets avec CSS Grid.

• Utiliser gap, flex-wrap, grid-template-columns et grid-column.

• Adapter une page aux écrans mobile, tablette et ordinateur.

10. Conclusion

Ce TP m'a permis de comprendre comment Flexbox et CSS Grid facilitent l'organisation d'une page Web. J'ai également appris à utiliser les Media Queries pour adapter la mise en page aux mobiles, aux tablettes et aux ordinateurs. Le Responsive Design améliore ainsi la lisibilité et l'utilisation du site sur différents appareils.